DEVELOPER & DEPLOYMENT GUIDE

Petrol PumpAI

Multi-tenant Petrol-Pump CRM & Forecourt Platform

A complete, self-hosted SaaS for petrol pumps: sales, khata, wholesale, cashier, stock, staff and accounts — AI-assisted, offline-capable, installable on any device.

Repository: github.com/mcgary6567-lab/petrol-pumpai

Live (SaaS): petrol-pumpai.fly.dev

Version: 1.0.0 · **Document:** v1

Contents

1 System overview
2 Technology stack
3 System size & footprint
4 Modules (multi-module system)
5 Roles & permissions
6 Architecture & data model
7 AI, WhatsApp & offline
8 Hosting compatibility
9 Deploy A — Fly.io (SaaS)
10 Deploy B — single pump (Docker/VPS)
11 Auto-deploy (CI/CD)
12 Play Store app (TWA)
13 Environment variables
14 Backups, security & scaling
15 Local development
16 Platform-owner operations

1 · System overview

Petrol PumpAI is a single codebase that runs in two modes from one flag (`SAAS_MODE`):

SaaS mode (SAAS_MODE=1)
One cloud server hosts many pumps. Public sign-up, 7-day free trial, activation gate after a one-time cash payment, and a platform super-admin. This is the deployment at petrol-pumpai.fly.dev.

Single-pump mode (default)
Classic on-prem install for one pump: first-run setup wizard, one tenant, runs on the pump's own server or a small VPS.

Every data row carries a `tenant_id` and the login token pins a user to exactly one pump, so the same code isolates one pump or thousands. The web app is a PWA: installable on Android, laptop and desktop, and it keeps working offline, syncing when the network returns.

> **In one line:** A Node.js + React app with a single SQLite file for data, packaged as one Docker image, deployable to any always-on host with a persistent disk.

2 · Technology stack

Layer	Technology	Why
Runtime	Node.js 22 (LTS)	Modern JS runtime; ships a built-in SQLite driver
Language	TypeScript 5.7 (backend & frontend)	Type-safe, fewer runtime bugs
Backend	Express 4 (REST API)	Simple, battle-tested HTTP server
Database	`node:sqlite` — a single SQLite file	Zero-admin, one file = full backup, fast
Auth	JWT (jsonwebtoken) + bcryptjs	Stateless tokens; hashed passwords & PINs
Validation	Zod	Every API input is schema-checked
Scheduler	node-cron	Nightly backups, reminders, automations
Push	web-push (VAPID)	Browser push notifications
AI (optional)	Anthropic Claude SDK *or* Google Gemini	Voice/photo entry, WhatsApp replies, insights
Frontend	React 18 + Vite 6	Fast SPA, instant dev reload
Styling	Tailwind CSS 3	Consistent, responsive UI
Routing	react-router-dom 6	Client-side navigation
Charts / icons	Recharts, lucide-react	Dashboards & crisp icons
QR	qrcode (generate), jsQR (scan)	Coupons, walk-in receipts, card scan
App shell	PWA (service worker + manifest)	Install + offline; Play Store via TWA
Messaging (optional)	WhatsApp Cloud API (Meta)	Two-way customer WhatsApp
Container	Docker (node:22-slim, multi-stage)	One image runs anywhere

> **No external database server needed.** There is no Postgres/MySQL/Redis to run. Data is one SQLite file on a persistent disk. This is why hosting is cheap and simple — but also why the host must give you a **persistent disk** and keep **one always-on process** (see §8).

3 · System size & footprint

Item	Size	Notes
Source code (TS/TSX)	~35,500 lines	50 backend route modules + full React app
Repository (no node_modules)	~61 MB	Includes built web assets & screenshots
Built frontend (web/dist)	~26 MB	Main JS bundle ~1.9 MB (≈0.5 MB gzipped)
Docker image	~250–350 MB	node:22-slim base + app
Database (empty)	< 1 MB	Grows with real data
Database (demo: 40,000+ sales, 37 customers)	~8 MB	Years of a busy pump fit in tens of MB
RAM at runtime	~150–300 MB	1 GB machine is comfortable
Persistent disk	3 GB is plenty to start	DB + nightly backups; extend anytime

> **Rule of thumb:** one small always-on machine (1 shared CPU, 1 GB RAM) + a 3 GB volume runs a busy pump — or a whole SaaS of many pumps to start. Scale RAM/disk as pumps grow; never run two writers (SQLite is single-writer).

4 · Modules (multi-module system)

The API is organised into 50 route modules. Functionally they group as follows. Each module is permission-gated per role (§5) and multi-tenant (§6).

Sales & shifts

POS sale (petrol/diesel/hi-octane, cash/card/khata) · nozzle meter readings & test litres · shift open/close with online-total reconciliation · bookings.

Customers & khata (credit)

Customer records & vehicles · khata ledger with per-customer limit & fixed discount · prepaid coupons & wallets · pending-card khata · WhatsApp inbox & CRM · orders · complaints · campaigns.

Wholesale supply & carriage

Wholesale clients with own rate card & running account · order book · tanker trips & fleet/drivers · recovery (wasooli) call-list & reminders · carriage/kiraya (thekedar) with per-trip settlement.

Money, cashier & accounts

Cashier desk (receive/pay, day book) · cheques (clear/bounce) · cash & bank with Pakistani bank list + per-bank tally · bank-statement reconciliation · card-machine (POS) settlement · expenses by category · suppliers & their ledger · property/rent · other income/expense · tax & accounts.

Stock, rate & shop

Tanks & live stock with stock-out prediction · stock register (received/sold/balance) · wet-stock reconciliation (dip vs book) · fuel price management · shop & lubricants mini-POS.

Staff & safety

Staff accounts · attendance (selfie + geolocation) · duty roster · salary, advances & loans · ratings & commission · licences & daily safety checklist · machines register (service, faults, warranty).

Owner control, corrections & AI

Owner dashboard & insights · monthly PDF report · margins & targets · profit explainer · **corrections (void)** — the CEO can reverse any wrong entry leg-for-leg so the books stay balanced (0-rupee difference) · full audit log (who changed what, before/after) · AI automations · users & roles · admin-managed lists & settings.

Using modules in code

Each module is an Express `Router` in `server/src/routes/*.ts`, mounted under `/api` in `server/src/index.ts`. Every authenticated request runs `requireAuth → tenantGate (SaaS) → permission check → auditTrail`. Helper `tid(req)` returns the caller's `tenant_id`; all queries filter by it. The React app mirrors modules as pages under `web/src/pages/`, navigated from `web/src/components/Layout.tsx`.

5 · Roles & permissions

Role	Sees / does
Admin (CEO / Malik)	Everything: reports, settings, users, prices, and **corrections/void**. The only role that can reverse wrong entries.
Manager	Day-to-day operations: rates, stock, approvals, staff.
Cashier	Receive/pay money, cheques, bank, and salesman cash handover; day book.
Salesman	Own sales, shift, meter readings, daily checks.
Wholesale officer	Wholesale supply, tanker trips, rates, recovery.

Permissions are fine-grained strings (e.g. `sales.create`, `khata.manage`, `corrections.manage`, `cashier.desk`). The UI and the API both enforce them; the token carries the user's role, tenant and resolved permissions. Staff can sign in with email+password or a quick 4-digit device PIN.

6 · Architecture & data model

```
      Browser / installed PWA (React + Vite + Tailwind)
                    |  HTTPS (JSON REST)
          Express API (Node 22 + TypeScript)
  requireAuth -> tenantGate -> permissions -> auditTrail
                    |
        node:sqlite  (one file: /data/pumpai.db)
                    |
    node-cron: nightly backup, reminders, automations
```

Multi-tenant isolation

- Every table has a `tenant_id`; every query filters by the caller's tenant.
- The JWT pins a user to one `tenant_id` — a user can never read another pump's data.
- **SaaS lifecycle:** a tenant is `trial` / `active` / `suspended`. The `tenantGate` middleware returns HTTP 402 and locks the pump to the Activate screen when a trial expires or a pump is suspended, until the owner enters an activation code or the platform owner flips it to active.
- **Derived-ledger corrections:** reversing a sale mirrors the original postings leg-for-leg (stock, cash, khata), so totals always reconcile — the Hisaab check proves 0 difference.

Database migrations

Schema is created and migrated in code on boot (`server/src/db.ts` → `migrate()`, idempotent `addColumn` / `CREATE TABLE IF NOT EXISTS`). No separate migration tool to run — starting the server upgrades the schema safely.

7 · AI, WhatsApp & offline

AI (optional, pluggable)

Set one key and the app uses it; set none and it falls back to a built-in rule engine (still fully usable).

- `ANTHROPIC_API_KEY` → Claude. `GEMINI_API_KEY` → Google Gemini. Claude wins if both are set.
- Powers: **photo meter reading**, **voice entry** (Roman Urdu), WhatsApp auto-replies, "ask your business anything", and insights.
- One platform key can serve all pumps, or each pump sets its own in Settings → Integrations.

WhatsApp (optional)

WhatsApp Cloud API (Meta): `WA_TOKEN`, `WA_PHONE_NUMBER_ID`, verify token & app secret. Without these, outbound messages are stored as "simulated" so the app still works end-to-end.

Offline & install

A service worker caches the app shell; the POS and entry flows work with no internet and sync when back online. "Install" adds it to a phone home screen or as a desktop app. The same PWA is wrapped as a **TWA** for Google Play (§12).

8 · Hosting compatibility

The app needs two things: **(a) one always-on Node process** and **(b) a persistent disk** for the SQLite file. Any host that gives both will run it; hosts that don't, won't.

Host type	Works?	Why
VPS (Ubuntu + Docker) — Contabo, Hetzner, DigitalOcean, Linode, AWS EC2	YES	Full control, always-on, real disk. Ideal for self-host.
Fly.io	YES	Docker + persistent volume + always-on machine. Used for the SaaS.
Railway / Render (with a disk/volume)	YES	Docker + attached persistent volume, always-on.
Any Docker host / your own server / mini-PC	YES	One container + a mounted data folder.
Shared hosting / cPanel (Hostinger shared, GoDaddy shared, Bluehost)	NO	PHP/static only; cannot run a long-lived Node process. (Hostinger **VPS** is fine — shared is not.)
Vercel / Netlify / Cloudflare Pages (serverless)	DEMO ONLY	No always-on process & ephemeral disk — SQLite resets. Fine for a throwaway demo, not for real data.
Static/object storage (S3, GitHub Pages)	NO	No server at all.

> **Decision rule:** if the host advertises "shared hosting", "serverless" or "static", it will **not** keep your database. Choose a **VPS** or a platform with a **persistent volume** (Fly.io, Railway, Render).

Cheapest real options: Fly.io ~$5–8/mo · small VPS ~$4–6/mo

Domain + SSL: Any registrar; SSL is free & automatic on Fly.io / Caddy / Let's Encrypt

9 · Deploy A — Fly.io (SaaS, many pumps)

One cloud server, public sign-up, trial + activation gate, platform admin. Config lives in `fly.toml`; full notes in `SAAS.md`.

```
fly auth login
fly apps create petrol-pumpai
fly volumes create pumpai_data --size 3 --region sin -a petrol-pumpai

# secrets (JWT_SECRET must never change after launch)
fly secrets set -a petrol-pumpai \
  JWT_SECRET="$(openssl rand -hex 32)" \
  PLATFORM_ADMIN_EMAIL="you@yourmail.com" \
  PLATFORM_ADMIN_PASSWORD="a-strong-password" \
  PUBLIC_URL="https://petrolpumpai.site" \
  SALES_PHONE="923443439061"

fly deploy -a petrol-pumpai
fly certs add petrolpumpai.site -a petrol-pumpai   # then add the DNS records shown
```

Region: `sin` (Singapore) — low latency to Pakistan

Machine: 1× shared-cpu-1x, 1 GB RAM, always-on (min_machines_running=1, auto_stop=false)

Data: Volume mounted at `/data` → DB at `/data/pumpai.db`, backups at `/data/backups`

Platform panel: `https://<domain>/platform` — activate/suspend pumps, make activation codes

> **Why always-on:** SQLite is a single writer and the scheduler runs nightly backups, reminders and automations — the machine must not auto-stop, and you must keep exactly one machine (never `fly scale count 2`).

10 · Deploy B — single pump (Docker / VPS)

For one pump on its own server or a small VPS. `SAAS_MODE` stays off → first-run setup wizard, one tenant.

With Docker (any VPS)

```
git clone https://github.com/mcgary6567-lab/petrol-pumpai
cd petrol-pumpai
docker build -t petrolpumpai .
docker run -d --name petrolpumpai --restart unless-stopped \
  -p 80:4000 \
  -v /srv/pumpai-data:/data \
  -e NODE_ENV=production \
  -e DB_PATH=/data/pumpai.db \
  -e BACKUP_DIR=/data/backups \
  -e JWT_SECRET="$(openssl rand -hex 32)" \
  -e PUBLIC_URL="https://yourpump.com" \
  petrolpumpai
```

The data folder `/srv/pumpai-data` on the host holds the database and backups — that folder is your entire install to back up. Put Nginx/Caddy in front for HTTPS (Caddy gives automatic Let's Encrypt SSL).

> **First run:** open the site, the setup wizard asks for the pump name, tanks, prices and the admin account, plus a setup code printed in the server logs (`[setup] SETUP CODE: …`).

11 · Auto-deploy (CI/CD)

A GitHub Actions workflow (`.github/workflows/fly-deploy.yml`) deploys on every push to `main`. One-time setup:

```
fly tokens create deploy -a petrol-pumpai
# In the GitHub repo:
#  Settings -> Secrets and variables -> Actions
#   New secret   : FLY_API_TOKEN = <the token>
#   New variable : FLY_APP       = petrol-pumpai
```

After that, every `git push` builds the image and deploys automatically; watch runs at `github.com/<owner>/<repo>/actions`.

Testing & build commands

Backend tests: `cd server && NODE_ENV=test npx tsx --test test/*.test.ts` (520+ tests)

Type check: `npx tsc --noEmit` (server & web)

Web build: `cd web && npm run build` → `web/dist`

12 · Play Store app (TWA)

The PWA is wrapped as a Trusted Web Activity — a thin Android shell that opens your live site full-screen. Full steps in `PLAYSTORE.md`.

```
npm i -g @bubblewrap/cli
bubblewrap init --manifest https://petrolpumpai.site/manifest.webmanifest
bubblewrap build        # prints the signing key SHA-256

# tell the server the app identity (serves /.well-known/assetlinks.json)
fly secrets set -a petrol-pumpai \
  ANDROID_PACKAGE="site.petrolpumpai.twa" \
  ANDROID_CERT_FINGERPRINTS="<sha256>,<play-signing-sha256>"
```

Upload the `.aab` to Play Console, set the privacy URL `/privacy`, and submit. You rarely rebuild the app — pushing code updates the website and the app shows it instantly.

13 · Environment variables

Variable	Required	Purpose
`JWT_SECRET`	Yes (prod)	Signs login tokens. Random 32+ bytes; never change after launch.
`PUBLIC_URL`	Yes	The address customers open (links, bills, app).
`DB_PATH`	Yes	SQLite file path on the persistent disk, e.g. `/data/pumpai.db`.
`BACKUP_DIR`	Recommended	Nightly backup folder, e.g. `/data/backups`.
`SAAS_MODE`	SaaS only	`1` = multi-tenant (sign-up + trial + platform). Off = single pump.
`TRIAL_DAYS`	SaaS	Free-trial length (default 7).
`PLATFORM_ADMIN_EMAIL` / `PLATFORM_ADMIN_PASSWORD`	SaaS	Super-admin login at `/platform` (seeded every boot).
`SALES_PHONE`	SaaS	WhatsApp number shown for buying/activation.
`ANTHROPIC_API_KEY` / `GEMINI_API_KEY`	Optional	Enable AI; else rule engine.
`WA_TOKEN`, `WA_PHONE_NUMBER_ID`, `WA_VERIFY_TOKEN`, `WA_APP_SECRET`	Optional	WhatsApp Cloud API.
`ANDROID_PACKAGE` / `ANDROID_CERT_FINGERPRINTS`	Play Store	Digital Asset Links for the TWA.
`BACKUP_COPY_CMD`	Optional	Command to copy each nightly backup off the server.

14 · Backups, security & scaling

Backups

- The scheduler writes a nightly DB backup into `BACKUP_DIR` (on the same volume, so it survives restarts).
- SQLite = one file: copying `pumpai.db` is a full backup. Keep a copy **off** the server (set `BACKUP_COPY_CMD`, or `fly ssh` / `docker cp` on a schedule).

Security

- Passwords & PINs hashed with bcrypt; JWT-signed sessions; per-role permissions enforced on every request.
- Security headers & rate limiting on auth; strict input validation (Zod).
- Full audit log of every change (who, when, before/after). Tenants are fully isolated by `tenant_id`.
- Keep `JWT_SECRET` and admin credentials in the host's secret store — never in the repo.

Scaling

- Grow vertically: `fly scale memory 2048`, extend the volume (`fly volumes extend`). One machine only — SQLite is single-writer.
- SQLite comfortably handles a busy pump and a large SaaS to start; migrate to a client/server DB only if write concurrency ever becomes the bottleneck.

15 · Local development

```
git clone https://github.com/mcgary6567-lab/petrol-pumpai
cd petrol-pumpai

# backend (demo data loads automatically in dev)
cd server && npm install
DEMO_DATA=1 npm run dev        # API on :4000

# frontend (separate terminal)
cd web && npm install && npm run dev   # Vite on :5173
```

Demo login: `admin@pumpai.pk` / `demo1234` (plus manager/salesman/cashier/wholesale demo accounts). In production the first-run wizard (or SaaS sign-up) creates real accounts.

16 · Platform-owner operations (SaaS)

- **Sign in:** `https://<domain>/platform` with `PLATFORM_ADMIN_EMAIL` / `PASSWORD`.
- **Activate a pump** after a cash payment: flip it to `active`, or generate a one-time code (`PPAI-XXXX-XXXX`) and send it on WhatsApp — the owner enters it in the app.
- **Suspend** a pump to lock it; **list** all pumps and their status.
- Change a forgotten super-admin password: `fly secrets set PLATFORM_ADMIN_PASSWORD="…"` (re-seeded on next boot).

> **Summary:** One Node + React + SQLite app, one Docker image, one always-on machine with a persistent disk. Deploy to Fly.io or any VPS, point a domain at it, and you have a full multi-pump SaaS — AI-assisted, offline-capable, installable everywhere, with the owner always able to fix any wrong entry.